Studio 3

Inheritance

Objectives

To introduce the concept of inheritance, to design classes using inheritance and implement classes in Processing.

Exercise 1

1. Discuss the following 2 questions.

   What is a super class? What is a sub-class?

2. When declaring fields in a class discuss the differences between using the following keywords:

   public, private, protected

3. A university stores the following information about all students enrolled at the university:

   id, name, address, phone.

   All international students will also need their passport number and visa type and visa expiry date stored.

   Design the classes required to support this. Each class will need an appropriate constructor and a method to return back all information about the student as a string.

4. Create objects in the main sketch code and test your objects.

5. Create a method that returns back just the name of the student. Think carefully which class the method needs to go in and then write the code for the method.

6. Create a method which returns back just the visa expiry date. Think carefully which class the method needs to go in and then write the code for the method.

7. Test the method in the main sketch code.

8. Store student and international students in a collection and display their information to the console window.

Exercise 2

1. Open your button sketch from Studio 2. You are now going to check if a button has been clicked on display a message if it has.

2. In the button class create a method called isClicked which returns a boolean and is passed the current x and y position. Write the code so that it returns true if the x and y position of the mouse is inside the rectangle and false if it isn't. Discuss with the demo if you are unsure how to do this.

3. In the sketch code create the mousePressed method and write the code that loops through all the buttons in the button list and if the current button is clicked on then displays a message to the console window.

Exercise 3

1. Open your lollipop sketch from Studio 2. You are now going to make the lollipops magically grow in size.

2. In the Lollipop class remove the lollipop size constant and replace it with a private field. Change the constructor so that the size field is set to 60. Note: don't change the paramters of the constructor as you don't need pass the size in. Change any other code required to use the new size fields and test your sketch to check that everything still works ok.

3. In the Lollipop class create a method called growLollipop which doesn't need anything passed to it and doesn't return anything and it will increase the size of the lollipop by 1 pixel.

4. Go to the sketch code and at the bottom of the draw method write the code to check if the list is not empty. If it is not empty then generate a random number between 0 and the end of the lollipop list to generate a random index position and store it in a variable. Then make the lollipop at that position grow. Run the code and check that random lollipops start growing.

5. In the lollipop class create a method called changeColour which isn't passed anything and doesn't return a value. The method will change the lollipop's colour to another random colour. How would you change the constructor so that you don't have any repeated code when setting to a random colour?

6. Investigate the keyPressed method on the Processing website and then create the method in your sketch code. Then inside it write the code that will make all the lollipops in the list change colour.

Exercise 4

1. Create a new sketch. Download the sourceText.txt file from the Assignment1 briefs folder on Moodle and then find the file and drag and drop it into the processing window to add it to your sketch.

2. Create the setup method and set the size to something large.

3. Create a string array reference variable inside your setup method called lineArray.

```
String[] lineArray;
```

4. Investigate the loadStrings method on the processing website and use it to load all of the lines into lineArray.

5. Write a for loop that will loop through all of the lines in the array and print each line to the console window. Hint: use the [] notation to access an element in the array.

6. Comment out the for loop and it's code and use a for each loop to write all the lines to the console window.

7. Create an ArrayList at class scope level called wordlist which will store an individual word from the text in each element.

8. In the setup method create another string array reference variable called wordArray. This array will be used to split up the words on a line.

9. Comment out the for each loop and underneath it create another loop to loop through the elements in the lineArray. You can use a normal for loop or a for each loop, whichever one you prefer.

10. Investigate the split method on the Processing website and use it inside your loop to split the words up on the current line, storing the words into wordArray.

Note: Some words will probably contain punctuation so if that bothers you then you can figure out how to get rid of punctuation in the words.

11. After splitting the line use another for loop or for each loop to go through all the elements in wordArray and add each word to wordList.

12. At the bottom of the setup up outside any of the previous loops write the code to loop through all the words in wordList and display each word to the console window.

    You should now see all the words from the text file displayed as individual words in the console window.

13. At class scope level create a reference variable from the PFont class.

    ```
    PFont font;
    ```

    Also create a variable to store the size of the font which will be a whole number and set it 16;

14. Investigate the createFont method on processing and add the code to set the font object to Arial with a size of 16 and anti alias to on just after the size method call.

15. Create the draw method and set the fill colour to any colour of your choosing.

16. Investigate the textFont method on the processing website and in the draw method use it to set the font to Arial and set the size using your font size variable.

17. At class scope level create a variable called index which will be used to access the words in wordList and set it to 0.

18. In the draw method write an if statement that checks if you haven't reached the end of the wordList yet.

19. Investigate the text method on the Processing website and inside the if statement use it to display the current word in wordlist at the current x and y position of the mouse and then move on to the next word in the list.

20. Create the else part for the if statement that will go back to the beginning of wordList.

    Test your code. Notice the mouse position is the top left corner of the text, investigate the textAlign method on the Processing website and try different alignments.

Exercise 5 (Studio Handin Exercise)

1. Create a new Sketch and add the setup method. Download the lotto.csv file from the Studio section on Moodle and add it to your sketch.

2. Create a class called LottoLine which will be used to store the numbers on a line of a lotto ticket. Create a protected field to store the 6 lotto numbers on a line using an array of integers.

3. Create the constructor for the class which is passed the 6 lotto numbers as individual values and creates 6 elements for the number array and then stores the numbers into each position in the array.

4. Create the toString method that will loop through the array and join all the numbers together as one string with a space between each number. The string is then returned back from the method.

5. Create an array list to store lotto line objects inside the setup method.

6. In the setup method write the code to load all the lotto lines into an array as strings. Each element in the array will hold the 6 lotto numbers in csv format.

7. Now extend the code to loop through the line array and inside the loop split the csv line using another array and then check that the size of the array is exactly 6 elements, if it isn't then display an error message showing the data on the line to the console window.

8. Create 6 integer variables in the setup method to hold the 6 lotto numbers after extraction. If the array is the correct size then use the Integer.parseInt method to extract the 6 lotto numbers from the split array into the 6 variables.

   Then create a new LottoLine object and add it to the list.

9. After the data from the file has been loaded, loop through the list and display each lotto line object to the console window. Note: you will get errors for the powerball lines in the file.

10. Now create the PowerballLine class which will inherit from the LottoLine class. A powerball line stores the 6 lotto numbers as well as an additional powerball number. Add an extra field to store the powerball number.

11. Create the constructor for the PowerballLine class.

12. Create the toString method that will return back the 6 lotto numbers as well as the powerball number at the end of of the lotto nmbers. Hint: think carefully how to do this without repeating code from the LottoLine class.

13. Now change the setup method so that if the size of the split array is not 6 then it checks if the size is 7. If it is 7 then extract the seven values out and create a new PowerballLine object and add it to the list.

14. Check to make sure that lotto and powerball line data is being displayed in the console window.

Optional (not required for handin)

15. In the LottoLine class create a method called checkNumbers which is passed an array of integers (the winning numbers) and returns back the how many of the first 6 numbers in the array are in the lotto line. You can assume the array passed in will always have at least 6 elements.

    Work with the demonstrator or tutor to come up with the pseudo-code to solve this problem and then write the code.

16. Create the same method in the PowerballLine class which does the same thing but also checks if the 7th element in the array passed in matches the powerball number. Hint: Don't repeat the code from the LottoLine class, make your method elegant.

17. Now change your sketch code so that after displaying a lotto line or powerball line object it displays how many matching numbers there are underneath the line using an array with the following declaration:

```
int[] winNumsArray = new int[] {29,10,12,8,32,27,18};
```